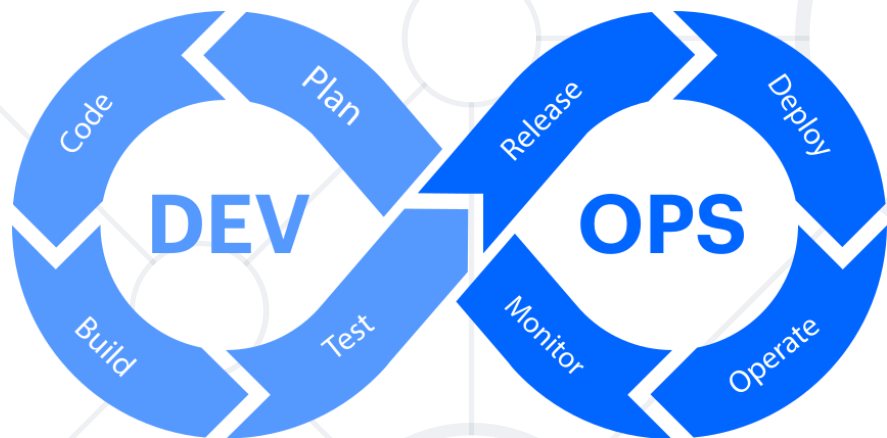


Release Automation

Packaging, Templating, Automated Deployment



SoftUni Team
Technical Trainers



SoftUni



Software University

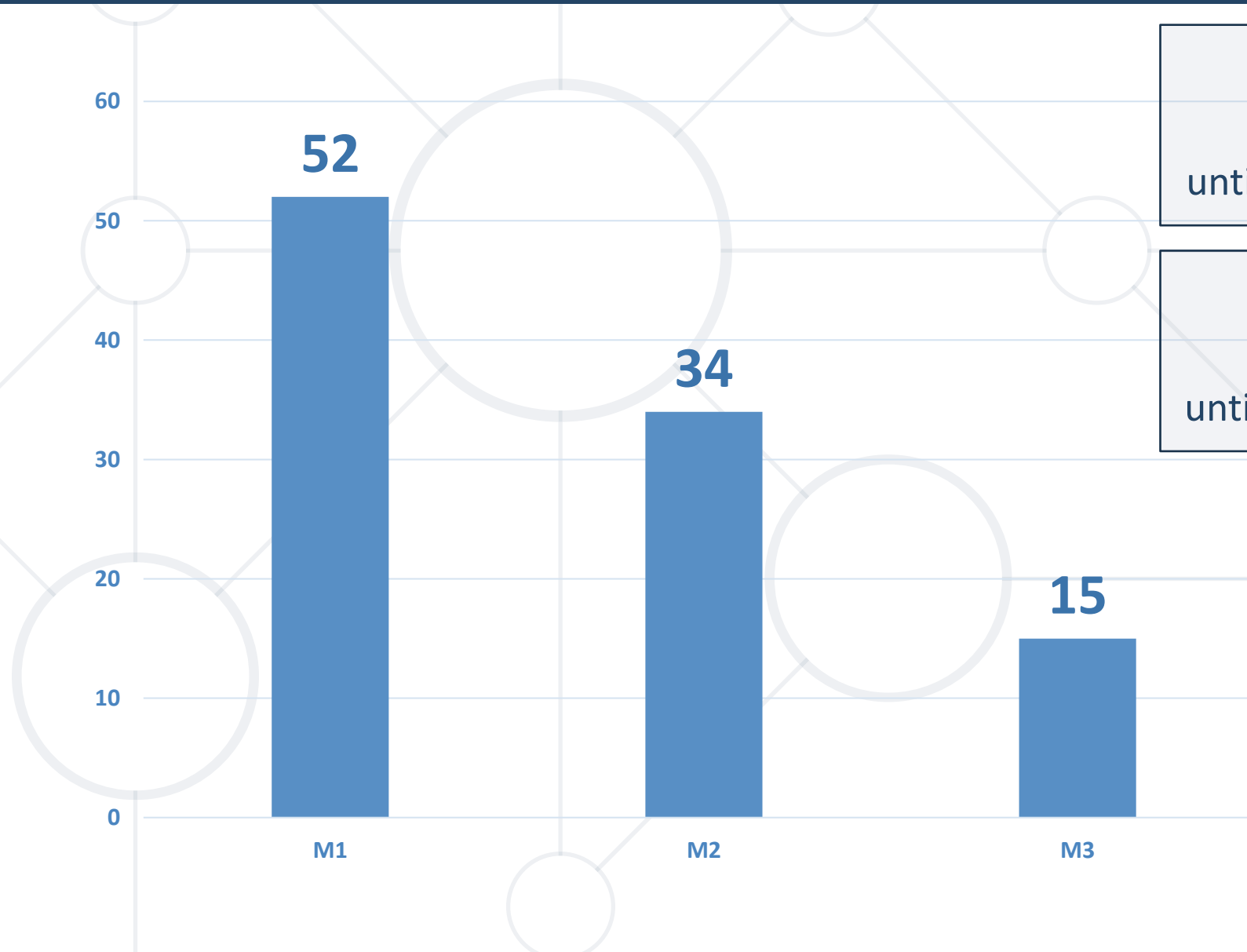
<https://softuni.bg>

sli.do

#DevOps-CI-CD

[facebook.com/groups/
cicdmonitoringjanuary2026](https://facebook.com/groups/cicdmonitoringjanuary2026)

Homework Progress



Solutions for M3
can be submitted
until 23:59 on 11.02.2026

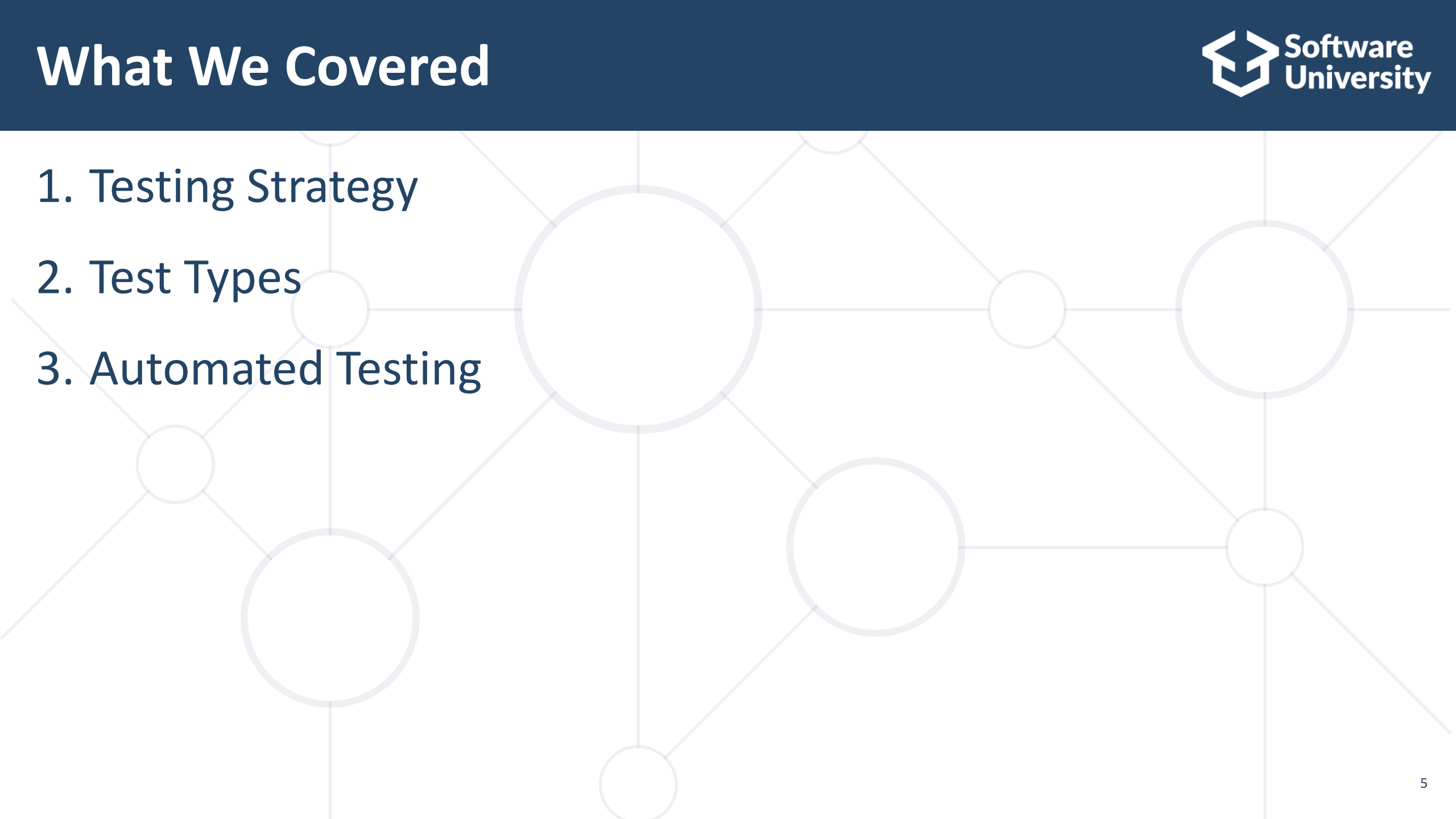
Solutions for M4
can be submitted
until 23:59 on 18.02.2026



Previous Module (M3)

Quick Overview

What We Covered

- 
1. Testing Strategy
 2. Test Types
 3. Automated Testing



This Module (M4)

Topics and Infrastructure

1. Packaging and Templating
2. Deployment and Organizational Strategies
3. Enable Automated Deployment

Crawl
(Docker Compose)

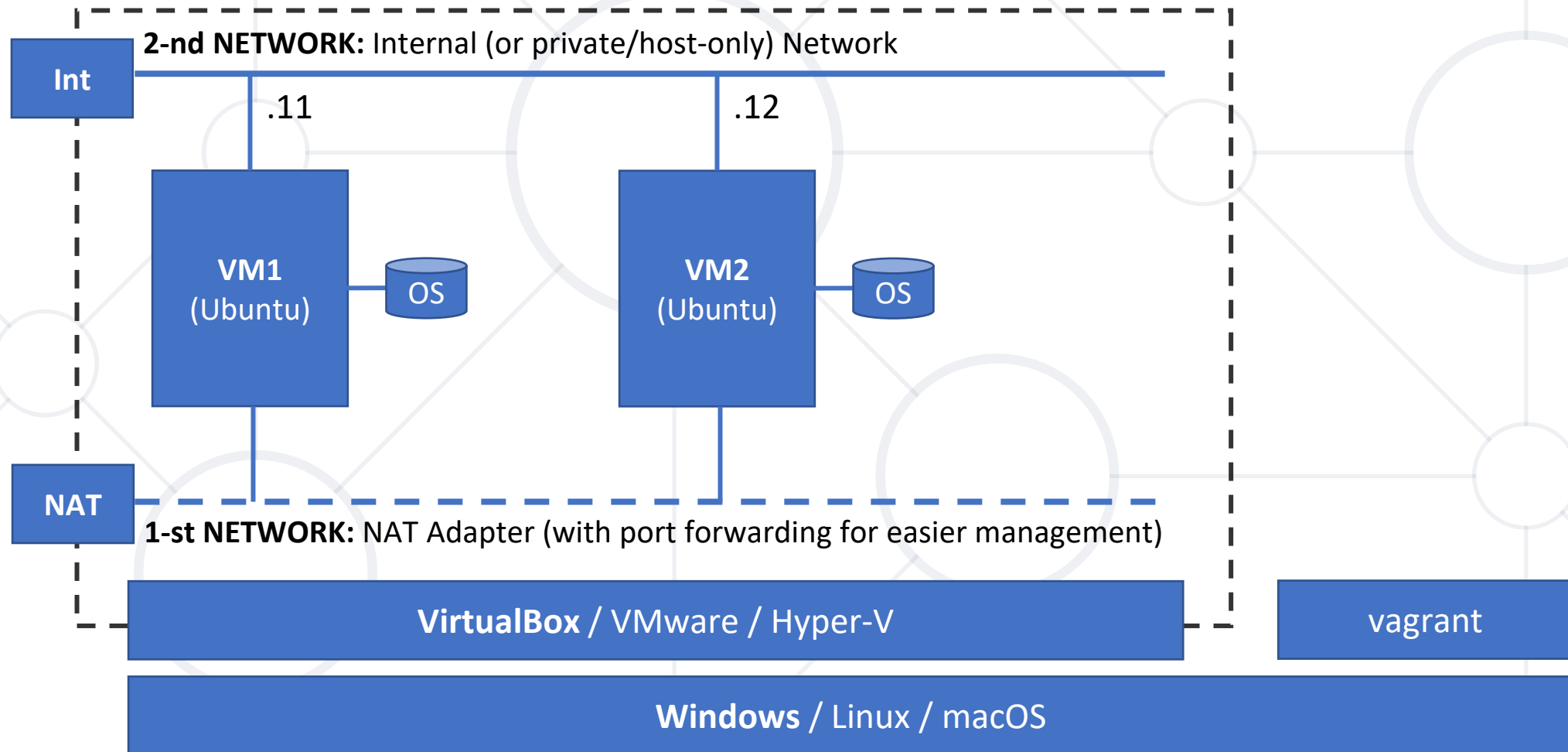


Walk
(Kubernetes Manifests)



Run
(Kubernetes and Helm)







Docker Compose

Transition from Local to CI/CD

- **We must make decisions** and form strategies not only when it comes to deployment, **but on every** other **stage** as well
- **Application architecture** (monolith, n-tier, microservices, etc.)
- **Repository structure** (mono or poly)
- **Pipelines** (specialized per component or phase, or all-in-one)
- **Packaging**/templating approach (separate, or in groups)
- **Deployment** (in terms of batches, and actual delivery)

- A tool for **defining** and **running multi-container applications**
- Provides **streamlined and efficient** development and deployment experience
- Manage **services, networks, and volumes** in a single YAML file
- Supports **multiple environments** via separate files, profiles and parametrization (via .env files)
- Mostly **suitable for single/local machine**

- Instead of executing manual docker compose commands
- We could go a step further and automate the process
- Delegate this to our pipeline(s) – add the CD to the pipeline
- Key moments here are
 - How pipeline steps are executed – shell sessions, isolated containers, etc.
 - What is the architecture of the runner/executor
 - Where is the target deployment environment



Practice: Knowledge is the Key
Get Familiar with the Architecture 😊



Kubernetes and Static Manifests

Scaling and Management

- Nowadays, the target environment is usual a **Kubernetes** cluster
- Thus, we shift from bare containers to different **abstractions**
- **Workload** objects like **Pod**, **Deployment**, **StatefulSet**, etc.
- Communication-related objects like **Service**, **Ingress**, etc.
- And supplementary objects like **ConfigMap** and **Secret**
- All of the above is usually managed via **YAML manifests**
- A manifest could contain **single** or **multiple objects**
- Values like tags, ports, etc., are part of them and are hardcoded

- **Updating** a component or the application **requires extra work**
- One way, is to **adjust a manifest** and send it to the cluster
- Another option, is to **patch the image** in real-time
- Each has its pros and cons
- **Not all changes are equal** – image vs port (for example)
- Nice to have a way to **track the reason for the change**

- **Service discovery and naming**
- The same open questions from before
 - How pipeline steps are executed – shell sessions, isolated containers, etc.
 - What is the architecture of the runner/executor
 - Where is the target deployment environment



Practice: Tests
Let's Create a Few



Helm
The Production Standard

- Describe even the most complex apps, provide repeatable application installation, and serve as a single point of authority
- In-place upgrades and custom hooks
- Easy to version, share, and host on public or private servers
- Roll back to an older version of a release with ease
- Decoupling the **Logic** (Templates) from the **Data** (Values)

- **Single (umbrella) chart vs independent charts**
- Updating a component in an umbrella chart, use **--reuse-values**
- **All-or-nothing** upgrade with **--atomic**
- **Rollback** via command line or dedicated pipeline

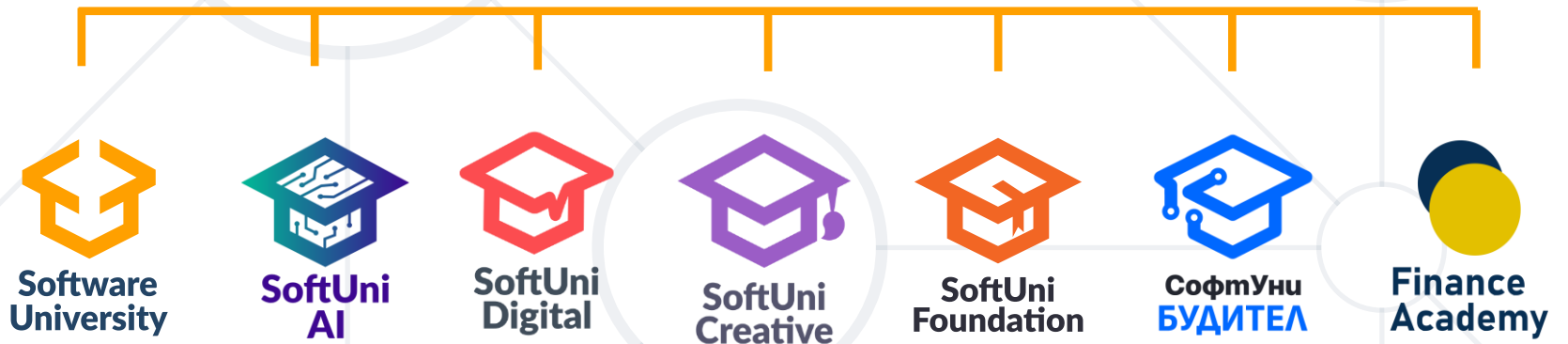


Practice: Automation
Let's Integrate Them in the Pipeline

Questions?



SoftUni



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg, about.softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

